

CrewAI

Role-Based Multi-Agent Orchestration Framework

CrewAI is an open-source framework designed to build, orchestrate, and manage collaborative AI agents that work together to accomplish complex tasks. Instead of relying on a single AI model to handle an entire workflow, CrewAI enables multiple specialized agents to cooperate, communicate, and delegate responsibilities, much like a real-world team. CrewAI follows the concept of **Agentic AI**, where autonomous agents can reason, make decisions, use tools, and collaborate with other agents to achieve a common objective. Each agent is assigned a specific role, goal, and expertise, allowing the system to break down complex problems into manageable subtasks.

Components

Component	Description
LLM	Wrapper that connects a CrewAI agent to a language model backend (e.g., Groq-hosted Llama 3.1).
Agent	An LLM-backed persona defined by a role, a goal, and a backstory; the basic unit of reasoning.
Task	A unit of work with a description and an expected_output, assigned to exactly one agent.
Crew	Groups a set of agents and tasks together and executes them under a chosen Process.
Process	The execution strategy for a crew — Sequential (ordered hand-off) or Hierarchical (manager-delegated).
kickoff()	The method that runs the crew end-to-end and returns the final result.
Tools (external)	Plain Python functions/libraries (e.g., SQLite, FPDF) combined with a crew for hybrid AI + business logic.

Syntax

```
# 1. Configure the LLM
from crewai import LLM
llm = LLM(model="groq/llama-3.1-8b-instant", api_key=GROQ_API_KEY)

# 2. Define an Agent
from crewai import Agent
agent = Agent(
    role="<role name>",
    goal="<what this agent is trying to achieve>",
    backstory="<persona/context for the agent>",
    llm=llm,
    verbose=True
)
```

```
# 3. Define a Task
from crewai import Task
task = Task(
    description="<what to do>",
    expected_output="<what a correct answer looks like>",
    agent=agent
)

# 4. Group into a Crew and run
from crewai import Crew, Process
crew = Crew(
    agents=[agent],          # list of Agent objects
    tasks=[task],            # list of Task objects (run in this order)
    process=Process.sequential,
    verbose=True
)
result = crew.kickoff()
print(result)
```

Learning Outcomes

- Build CrewAI Agents (role, goal, backstory) and Tasks (description, expected_output).
- Orchestrate single- and multi-agent Crews using Process.sequential.
- Explain how CrewAI automatically passes context between sequential tasks.
- Integrate a Groq-hosted LLM into a CrewAI agent pipeline.
- Combine AI agents with deterministic business logic and SQLite persistence.
- Ground agent reasoning in retrieved historical data (a RAG-style pattern).
- Generate a PDF report from a CrewAI pipeline's final output.

Software Requirements

Requirement	Detail
Python Version	3.10 or higher
Core Package	crewai==0.80.0
LLM Backend	langchain-groq==0.2.1
Other Libraries	rich (console output), fpdf (PDF generation), sqlite3 (stdlib, persistence)
API Key	GROQ_API_KEY — free at https://console.groq.com
Environment	Google Colab or any local Jupyter/VS Code setup
Install Command	pip install crewai==0.80.0 langchain-groq==0.2.1 fpdf rich

Lab Programs

Lab 1: Single-Agent CrewAI — Travel Researcher

Source Code

```
# =====
# LAB 1 -- Single-Agent CrewAI: Travel Researcher
# =====

from crewai import LLM, Agent, Task, Crew, Process

# -- LLM CONFIG (Groq-hosted Llama 3.1) -----
llm = LLM(
    model="groq/llama-3.1-8b-instant",
    api_key=GROQ_API_KEY
)

# -- AGENT -----
travel_researcher = Agent(
    role="Travel Researcher",
    goal="Find the best tourist attractions in Goa",
    backstory="You are an experienced travel expert.",
    llm=llm,
    verbose=True
)

# -- TASK -----
research_task = Task(
    description=""
    Find 5 popular tourist places in Goa.
    Mention why tourists visit them.
    "",
    expected_output="List of 5 tourist attractions with explanations.",
    agent=travel_researcher
)

# -- CREW (single agent, single task) -----
crew = Crew(
    agents=[travel_researcher],
    tasks=[research_task],
    process=Process.sequential,
    verbose=True
)

result = crew.kickoff()

print("\nFINAL RESULT:\n")
print(result)
```

Sample Input / Output

```
Agent: Travel Researcher
Task: "Find 5 popular tourist places in Goa..."

--- Output ---
FINAL RESULT:
1. Baga Beach - known for water sports and nightlife.
2. Basilica of Bom Jesus - UNESCO heritage church.
3. Dudhsagar Falls - one of India's tallest waterfalls.
4. Fort Aguada - 17th-century Portuguese fort with sea views.
5. Anjuna Flea Market - famous Wednesday market...
```

Key Concepts Covered

Concept	Description
Agent	An LLM-backed persona defined by role, goal, and backstory.
Task	A unit of work with a description and an expected_output, assigned to one agent.
Crew	Orchestrates agents and tasks under a chosen Process.

Lab 2: Student Career Advisor System (3-Agent Sequential Crew)

Source Code

```
# =====
# LAB 2 -- Student Career Advisor System (3 Agents, Sequential)
# =====

from crewai import Agent, Task, Crew, Process, LLM

# -- LLM -----
llm = LLM(
    model="groq/llama-3.1-8b-instant",
    api_key=GROQ_API_KEY
)

# -- AGENT 1: Skill Analyzer -----
skill_analyzer = Agent(
    role="Skill Analyzer",
    goal="Analyze student skills and strengths",
    backstory="You are an expert in evaluating student abilities.",
    llm=llm,
    verbose=True
)

# -- AGENT 2: Career Advisor -----
career_advisor = Agent(
    role="Career Advisor",
    goal="Suggest suitable career paths",
    backstory="You help students choose careers.",
    llm=llm,
    verbose=True
)

# -- AGENT 3: Learning Mentor -----
learning_mentor = Agent(
    role="Learning Mentor",
    goal="Create a learning roadmap",
    backstory="You design study plans and learning paths.",
    llm=llm,
    verbose=True
)

# -- TASK 1: Analyze the student's profile -----
analysis_task = Task(
    description="""
    Student Details:

    Skills:
    - Python
    - SQL
    - Communication

    Interests:
    - Artificial Intelligence
    - Data Science

    Analyze strengths and weaknesses.
    """,
    expected_output="Detailed skill analysis.",
    agent=skill_analyzer
)

# -- TASK 2: Recommend careers (uses Task 1's context) -----
career_task = Task(
    description="""
    Based on the student's skills and interests,
```

```

        recommend suitable career options.
        """
        expected_output="List of career recommendations.",
        agent=career_advisor
    )

# -- TASK 3: Build a learning roadmap (uses Task 1 & 2 context) -----
roadmap_task = Task(
    description="""
    Create a 6-month roadmap for the student
    to become job-ready.
    """,
    expected_output="Detailed learning roadmap.",
    agent=learning_mentor
)

# -- CREW: 3 agents, 3 tasks, sequential hand-off -----
crew = Crew(
    agents=[skill_analyzer, career_advisor, learning_mentor],
    tasks=[analysis_task, career_task, roadmap_task],
    process=Process.sequential,
    verbose=True
)

result = crew.kickoff()

print("\nFINAL RESULT:\n")
print(result)

```

Sample Input / Output

Skills: Python, SQL, Communication
Interests: AI, Data Science

```

--- Output ---
FINAL RESULT:
6-Month Roadmap:
Month 1-2: Python + SQL mastery, basic statistics.
Month 3-4: Machine learning fundamentals, mini projects.
Month 5-6: Portfolio projects, internship applications...

```

Key Concepts Covered

Concept	Description
Sequential Process	Process.sequential runs tasks in order, auto-passing context forward.
Multi-Agent Specialization	Each agent has a narrow role instead of one agent doing everything.
Implicit Context Passing	CrewAI threads each task's output into the next task's prompt.

Lab 3: Hospital Management System (4-Agent Sequential Crew)

Source Code

```
# =====
# LAB 3 -- Hospital Management System (4 Agents, Sequential)
# =====

from crewai import Agent, Task, Crew, Process, LLM

llm = LLM(
    model="groq/llama-3.1-8b-instant",
    api_key=GROQ_API_KEY,
    max_tokens=300
)

# -- AGENT 1: Receptionist -----
receptionist = Agent(
    role="Receptionist",
    goal="Register patients",
    backstory="You collect patient information.",
    llm=llm,
    verbose=True
)

# -- AGENT 2: Doctor -----
doctor = Agent(
    role="Doctor",
    goal="Diagnose patients",
    backstory="You are an experienced physician.",
    llm=llm,
    verbose=True
)

# -- AGENT 3: Pharmacist -----
pharmacist = Agent(
    role="Pharmacist",
    goal="Suggest medicines and precautions",
    backstory="You provide prescriptions and advice.",
    llm=llm,
    verbose=True
)

# -- AGENT 4: Billing Officer -----
billing_officer = Agent(
    role="Billing Officer",
    goal="Generate hospital bills",
    backstory="You manage patient billing.",
    llm=llm,
    verbose=True
)

# -- TASK 1: Register the patient -----
registration_task = Task(
    description="""
    Patient Details:

    Name: Ravi
    Age: 35

    Symptoms:
    - Fever
    - Headache
    - Body Pain

    Register the patient.
    """,
```

```

        expected_output="Patient registration summary.",
        agent=receptionist
    )

# -- TASK 2: Diagnose based on symptoms -----
diagnosis_task = Task(
    description="Analyze symptoms and provide diagnosis.",
    expected_output="Doctor diagnosis report.",
    agent=doctor
)

# -- TASK 3: Recommend medicines -----
medicine_task = Task(
    description="Suggest medicines and precautions.",
    expected_output="Prescription details.",
    agent=pharmacist
)

# -- TASK 4: Generate the bill -----
billing_task = Task(
    description="Generate consultation and medicine bill.",
    expected_output="Hospital bill.",
    agent=billing_officer
)

# -- CREW: 4 agents -> 4 tasks, sequential pipeline -----
crew = Crew(
    agents=[receptionist, doctor, pharmacist, billing_officer],
    tasks=[registration_task, diagnosis_task, medicine_task, billing_task],
    process=Process.sequential,
    verbose=True
)

result = crew.kickoff()

print("\nFINAL RESULT:\n")
print(result)

```

Sample Input / Output

```

Name: Ravi, Age: 35
Symptoms: Fever, Headache, Body Pain

--- Output ---
FINAL RESULT:
Hospital Bill
-----
Consultation Fee: Rs 500
Medicines: Rs 350
Total: Rs 850
Status: Payment Pending

```

Key Concepts Covered

Concept	Description
4-Stage Sequential Pipeline	Registration -> Diagnosis -> Medicine -> Billing, one agent per stage.
max_tokens	An LLM parameter limiting response length, tunable per use case.
Static Task Description	Patient details are hard-coded into the task description (no user input yet).

Lab 4: Capstone — Smart Hospital Management System

Source Code

```
# =====
# CAPSTONE -- Smart Hospital Management System
# CrewAI (4 Agents) + SQLite Persistence + PDF Report Generation
# =====

from crewai import Agent, Task, Crew, Process, LLM
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
from rich.rule import Rule
import sqlite3
from datetime import datetime, timedelta
from fpdf import FPDF

console = Console()

llm = LLM(
    model="groq/llama-3.1-8b-instant",
    api_key=GROQ_API_KEY,
    max_tokens=50
)

# -- DATABASE SETUP (SQLite) -----

def init_db():
    conn = sqlite3.connect("hospital.db")
    cur = conn.cursor()
    cur.execute("""
    CREATE TABLE IF NOT EXISTS patients(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        age INTEGER,
        symptoms TEXT,
        doctor TEXT,
        appointment_date TEXT,
        visit_date TEXT
    )
    """)
    try:
        cur.execute("ALTER TABLE patients ADD COLUMN visit_date TEXT")
    except Exception:
        pass
    conn.commit()
    conn.close()

init_db()

# -- HELPER FUNCTIONS (deterministic, non-LLM) -----

def book_appointment(patient_name, doctor_name):
    appointment_date = (datetime.now() + timedelta(days=1)).strftime("%d-%m-%Y")
    token = f"T{hash(patient_name) % 1000}"
    return appointment_date, token

def save_patient(name, age, symptoms, doctor, appointment_date):
    conn = sqlite3.connect("hospital.db")
    cur = conn.cursor()
    cur.execute("""
    INSERT INTO patients (name, age, symptoms, doctor, appointment_date, visit_date)
    VALUES (?, ?, ?, ?, ?, ?)
    """, (name, age, symptoms, doctor, appointment_date,
          datetime.now().strftime("%d-%m-%Y")))

```

```

conn.commit()
conn.close()

def get_patient_history(patient_name):
    conn = sqlite3.connect("hospital.db")
    cur = conn.cursor()
    cur.execute("""
        SELECT symptoms, doctor, appointment_date, visit_date
        FROM patients WHERE name = ? ORDER BY id DESC LIMIT 5
    """, (patient_name,))
    rows = cur.fetchall()
    conn.close()
    return rows

def generate_pdf(report_text):
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=12)
    pdf.multi_cell(0, 10, report_text)
    pdf.output("Patient_Report.pdf")

# -- USER INPUT -----

console.print(Panel.fit("[bold cyan]SMART HOSPITAL MANAGEMENT SYSTEM[/bold cyan]",
border_style="green"))

patient_name = console.input("Patient Name : ")
age = console.input("Age : ")
symptoms = console.input("Symptoms : ")

# -- HISTORY CHECK (returning vs new patient) -----

history = get_patient_history(patient_name)

if history:
    console.print(Panel.fit("[bold green]Returning Patient Detected[/bold green]",
border_style="green"))
    table = Table(title="Previous Visits")
    table.add_column("Visit Date")
    table.add_column("Symptoms")
    table.add_column("Doctor")
    for row in history:
        table.add_row(row[3], row[0], row[1])
    console.print(table)
else:
    console.print(Panel.fit("[yellow]New Patient[/yellow]", border_style="yellow"))

# -- DISPLAY PATIENT INFO -----

info = Table(title="Patient Information")
info.add_column("Field", style="cyan")
info.add_column("Value", style="green")
info.add_row("Name", patient_name)
info.add_row("Age", age)
info.add_row("Symptoms", symptoms)
console.print(info)

# -- BUILD HISTORY CONTEXT FOR THE AI AGENTS -----

history_text = ""
for row in history:
    history_text += f"""
Visit Date: {row[3]}
Symptoms: {row[0]}

```

```

Doctor: {row[1]}
"""

# -- AGENTS -----

symptom_agent = Agent(
    role="Symptom Analyzer",
    goal="Analyze symptoms and detect possible disease.",
    backstory="Experienced medical triage expert.",
    llm=llm,
    verbose=True
)

doctor_agent = Agent(
    role="Doctor Recommender",
    goal="Suggest correct specialist doctor.",
    backstory="Hospital doctor allocation system.",
    llm=llm,
    verbose=True
)

precaution_agent = Agent(
    role="Medical Advisor",
    goal="Suggest precautions and home care tips.",
    backstory="Senior healthcare advisor.",
    llm=llm,
    verbose=True
)

report_agent = Agent(
    role="Report Generator",
    goal="Generate final patient report.",
    backstory="Hospital documentation expert.",
    llm=llm,
    verbose=True
)

# -- TASKS (sequential, knowledge-grounded with patient history) -----

task1 = Task(
    description=f"""
Patient Name: {patient_name}
Age: {age}
Symptoms: {symptoms}

Previous History:
{history_text}

Analyze condition and severity.
""",
    expected_output="Disease prediction and severity.",
    agent=symptom_agent
)

task2 = Task(
    description=f"Symptoms: {symptoms}\nRecommend specialist doctor.",
    expected_output="Doctor specialization.",
    agent=doctor_agent
)

task3 = Task(
    description=f"Symptoms: {symptoms}\nGive precautions.",
    expected_output="Care instructions.",
    agent=precaution_agent
)

task4 = Task(

```

```

        description="Generate final medical report.",
        expected_output="Full structured report.",
        agent=report_agent
    )

# -- CREW EXECUTION -----

crew = Crew(
    agents=[symptom_agent, doctor_agent, precaution_agent, report_agent],
    tasks=[task1, task2, task3, task4],
    process=Process.sequential,
    verbose=True
)

console.print(Rule("[bold blue]AI Analysis Started[/bold blue]"))
result = crew.kickoff()

# -- BUSINESS LOGIC: booking + persistence (deterministic, non-LLM) -----

doctor_name = "General Physician"
appointment_date, token = book_appointment(patient_name, doctor_name)

save_patient(patient_name, age, symptoms, doctor_name, appointment_date)

# -- FINAL REPORT TEXT -----

final_report = f"""
PATIENT REPORT
-----
Name: {patient_name}
Age: {age}
Symptoms: {symptoms}

Doctor: {doctor_name}
Appointment: {appointment_date}
Token: {token}

AI ANALYSIS:
{result}

Status: Saved Successfully
"""

# -- PDF GENERATION + OUTPUT -----

generate_pdf(final_report)

console.print(Panel(final_report, title="[bold green]FINAL REPORT[/bold green]",
border_style="green"))
console.print(Panel.fit("PDF Generated: Patient_Report.pdf", border_style="green"))
console.print(Panel.fit("Saved to SQLite Database", border_style="green"))

```

Sample Input / Output

```

Patient Name: Ravi
Age: 35
Symptoms: fever, cough, fatigue

--- Output ---
Returning Patient Detected
+-----+-----+-----+
| Visit Date | Symptoms | Doctor           |
| 10-05-2026 | Fever    | General Physician |
+-----+-----+-----+

FINAL REPORT
Name: Ravi | Age: 35 | Symptoms: fever, cough, fatigue
Doctor: General Physician | Appointment: 25-06-2026 | Token: T482
AI ANALYSIS: Likely viral infection... recommend rest and fluids.
Status: Saved Successfully

PDF Generated: Patient_Report.pdf
Saved to SQLite Database

```

Key Concepts Covered

Concept	Description
SQLite Persistence	patients table stores every visit; queried for history on each new run.
Knowledge Grounding (RAG-style)	Retrieved patient history is injected into the agent's task description.
AI + Deterministic Hybrid Logic	LLM agents reason over symptoms; plain Python handles booking, IDs, and storage.
PDF Report Generation	FPDF turns the final text report into a downloadable Patient_Report.pdf.

Cheat Sheet

Syntax / Call	Purpose
<code>LLM(model=..., api_key=...)</code>	Create the LLM connection used by agents.
<code>Agent(role, goal, backstory, llm, verbose)</code>	Define an agent persona.
<code>Task(description, expected_output, agent)</code>	Define a unit of work for one agent.
<code>Crew(agents=[...], tasks=[...], process=...)</code>	Group agents + tasks for execution.
<code>Process.sequential</code>	Run tasks in order; auto-pass each output as the next task's context.
<code>Process.hierarchical</code>	A manager agent delegates tasks dynamically (not used in these labs).
<code>crew.kickoff()</code>	Execute the crew and return the final result.
<code>verbose=True</code>	Print step-by-step agent reasoning/logs to the console.
<code>max_tokens=N (on LLM)</code>	Cap the length of generated responses.
<code>console.input("...")</code>	Rich's prompt for interactive user input (capstone lab).

Industry Use Cases

Pattern	Real-World Application
Single-agent crew	Focused research/summarization assistants, content generators.
3-agent sequential crew	Career counselling platforms, automated advisory pipelines.
4-agent sequential crew	Hospital/clinic intake workflows, multi-stage approval processes.
Capstone hybrid system	EHR-integrated clinical assistants, customer service systems combining AI + database + document generation.

Troubleshooting

Issue	Solution
AuthenticationError from Groq	Check that GROQ_API_KEY is valid and not the placeholder string.
Dependency conflict warnings on pip install	Expected with crewai==0.80.0 pinned versions; safe to ignore unless imports fail.

Issue	Solution
Agent output shorter than expected	Increase max_tokens on the LLM() call.
sqlite3.OperationalError: no such column	Delete hospital.db and let init_db() recreate it, or rely on the ALTER TABLE migration.
Task output ignores prior task's context	Confirm process=Process.sequential is set; verify task order matches intended pipeline.

Conclusion

Across these four experiments, students moved from a single CrewAI agent to a capstone multi-agent system combining role-based reasoning with SQLite persistence and PDF reporting. CrewAI's core strength — expressing multi-agent pipelines with minimal boilerplate — was demonstrated alongside hybrid integration with deterministic Python logic for real-world reliability.